

AI Resume Screening System

Assignment II: Production Refactoring, API Design, and Quality Assurance

AIMLCZG546 - Software Engineering for Machine Learning
Group No. 179

Group details

Sl. No	BITS ID	Name	Qualitative Task Description	Percentage
1	2025AB05113	Prashant	Implemented ML training & preprocessing pipelines, built FastAPI REST endpoints, authored automated Pytest suite, and drafted initial report sections.	25%
2	2025AA05032	Prathap Wagle	Designed system architecture, integrated Streamlit UI & application layer, authored main report, added code comments, and validated submission readiness.	25%
3	2024AC05999	Prasanna R T	Conducted test design review, performed functional validation of candidate scoring workflows, and verified edge-case handling.	25%
4	2024AC05914	Pranav Mehrotra	Reviewed quality assurance evidence, conducted code audit reviews, and validated final submission package integrity.	25%

Submission statement. This submission evolves the working Assignment I system without removing its upload, explanation, human-oversight, or audit features.

Assignment compliance mapping

Requirement	Implementation	Verified evidence
1. Modular refactoring	ML, quality, shared application, audit, API, and UI modules	ml/ (including features.py), quality/, services/, app/, streamlit_app.py
2. Research vs production	Inline notebook prototypes for clean_text and TF-IDF compared with production modules	179.ipynb, Section 3
3. Logging and errors	INFO/WARNING/ERROR across six critical modules	logging_config.py, logs/
4. Formatting and linting	Recorded initial findings, then Black/isort/Flake8 pass	report/lint_before.txt and lint_after.txt
5. REST API	Startup model load; typed prediction, health, and metrics	app/api.py, app/schemas.py
6-7. Tests and ML tests	Unit, data, loss, persistence, inference, continuity, API	44 collected / 43 functions; report/pytest_verbose.txt
8. Quality metrics	Accuracy, F1, Brier, top-3, schema, balance, wired train/holdout length drift	report/metrics_snapshot.json
9. Production and security	Shadow/canary strategy and resume-data controls	Section 9

1. Continuity from Assignment I

Assignment II uses the same HR/recruitment problem, labelled job-role data, TF-IDF and logistic-regression pipeline, keyword explanation, human decision note, and measurable quality goals developed in Assignment I. The monolithic prototype is refactored into production modules and exposed through FastAPI.

Assignment I capability	Preserved Assignment II component
TXT/PDF/DOCX resume upload	ml/ingestion.py
Text normalization and classification	ml/preprocessing.py, ml/features.py, ml/trainer.py
Explanation and human oversight	services/scoring.py
Consistent submission orchestration	services/application.py
Audit history without raw resume storage	services/audit.py
Recruiter-facing dashboard	streamlit_app.py
New production inference contract	app/api.py and app/schemas.py

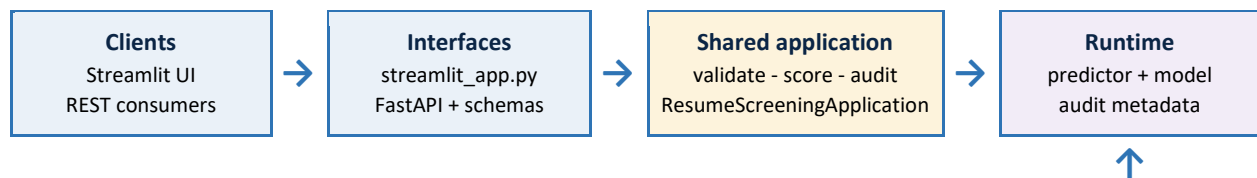
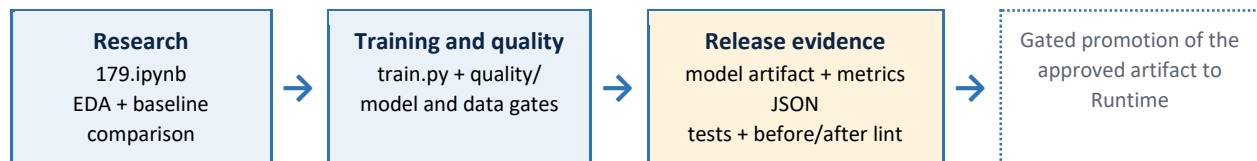
Online path - one shared request workflow**Offline path - evidence governs the release**

Figure 1. Shared online workflow and evidence-governed offline release. Horizontal arrows show request and training flow; the vertical arrow shows that an artifact reaches Runtime only after the offline quality gates and release evidence are satisfied.

2. Refactoring with OOP and functional principles

Component view (classes, modules, and key functions) verified against source imports and call paths. Streamlit and FastAPI both call the shared ResumeScreeningApplication; ResumeScoringService uses ModelPredictor and clean_text; train.py drives offline training through ml.data and ml.trainer with quality gates. Editable Mermaid source: docs/component_diagram.mmd.

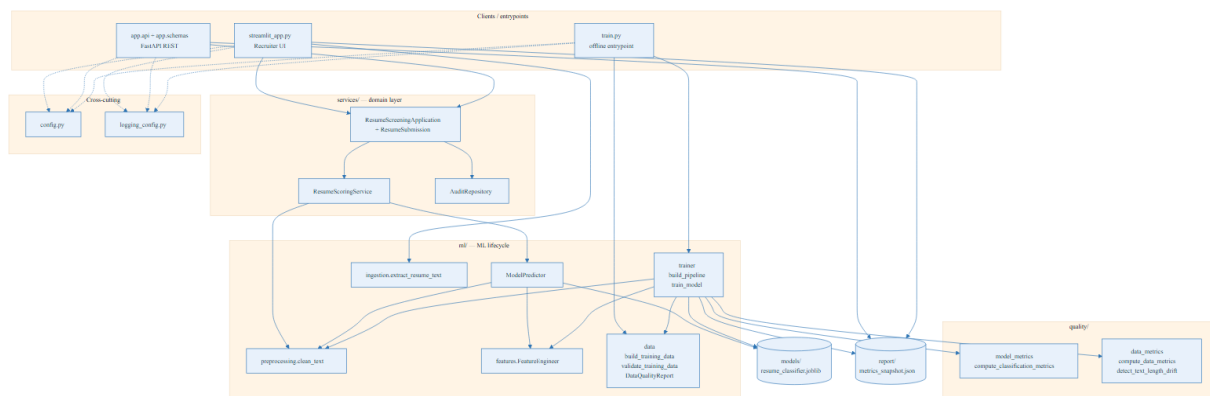


Figure 2. Production component overview of the modular Assignment II codebase: application entrypoints, shared services, ML lifecycle modules, quality checks, artifacts, and config/logging boundaries. Solid edges indicate runtime or training dependencies; dotted edges indicate cross-cutting configuration and logging.

Data and validation. ml/data.py creates the labelled dataset and returns an immutable DataQualityReport after schema, missing-value, duplicate, and class coverage checks.

Ingestion. ml/ingestion.py isolates TXT/PDF/DOCX parsing, file-type checks, empty-file checks, and a 5 MB upload limit.

Training. ml/features.py isolates TF-IDF as FeatureEngineer. ml/trainer.py builds the pipeline from that step plus logistic regression, measures validation metrics, enforces release gates, retrains on all data, and persists a versionable artifact.

Inference and services. ModelPredictor owns artifact loading and stable probability output. ResumeScoringService adds explanation and human oversight without coupling those rules to the classifier.

Application boundaries. ResumeScreeningApplication coordinates validate, score, and audit for both FastAPI and Streamlit. AuditRepository stores only metadata.

Quality and configuration. config.py centralizes runtime paths and versioning; quality/ computes model/data metrics and drift signals; training persists metrics JSON.

3. Research code vs production code

Research code is the exploratory notebook layer used to understand the problem, inspect data, try modelling choices, validate assumptions, and demonstrate that the approach is feasible before it is converted into maintainable software. In this submission, 179.ipynb serves that role by keeping the experiment narrative, EDA, prototype preprocessing, feature extraction trials, model comparison, classification report, and confusion matrix in one reproducible research record. It also compares MultinomialNB with logistic regression on one fixed split; both score 1.0, while logistic regression is retained for stable probabilities and deterministic production behavior.

The production package moves the validated notebook logic into reusable modules with typed boundaries, validation, logging, automated tests, and API/UI integration. The notebook remains the experimentation record, while production code becomes the executable system used by Streamlit, FastAPI, and training scripts.

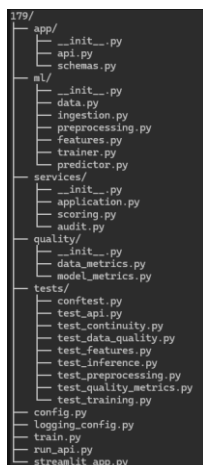


Figure 3. Research-to-production code separation in the submitted source package. The research notebook remains at the root as 179.ipynb, while reusable production logic is organized under app/, ml/, services/, quality/, and tests/.

Aspect	Research notebook path	Production package
Purpose	Explore feasibility, assumptions, metrics, and workflow behavior	Provide repeatable runtime and training behavior

Aspect	Research notebook path	Production package
Structure	Sequential cells combining explanation, prototype code, outputs, and demonstrations	Importable modules with explicit application, ML, service, quality, and test boundaries
ML workflow	EDA, inline preprocessing, TF-IDF trials, baseline comparison, validation report, and confusion matrix	Reusable pipeline, trainer, predictor, versioned artifacts, and metrics snapshot
Validation	Observed through notebook outputs, fixed holdout results, API demo cells, and audit preview	Enforced through data validation, request validation, release gates, and automated tests
Observability	Printed and displayed notebook outputs for interpretation	Structured logs, API responses, metrics endpoint, and persisted release snapshot
Quality evidence	Notebook execution demonstrates the research and demonstration path	Pytest, linting, metrics files, source zip, and runtime screenshots verify the package

This table summarizes the full notebook-to-package transition. In 179.ipynb, research code is represented by sequential cells for EDA, prototype preprocessing, feature extraction, baseline comparison, validation reports, API demonstration, audit continuity, and quality checks. In the production package, the same responsibilities are moved into reusable modules, persisted artifacts, tested contracts, structured logs, and API/UI entrypoints.

4. Error handling and logging

Logging is configured centrally and writes to console plus a rotating project-relative file (1 MB, three backups). Raw resume text is neither logged nor stored in the audit CSV.

Critical module	INFO	WARNING / ERROR
ml/preprocessing.py	Input/output character counts	Wrong type, empty, oversized, or unusable content
ml/ingestion.py	Type and extracted length	Unsupported, empty, oversized, or failed extraction
ml/data.py	Rows, schema, quality report	Missing schema/values or insufficient classes
ml/trainer.py	Training rows, artifact, metrics	Fit exception or failed quality gate
ml/predictor.py	Artifact and prediction metadata	Missing artifact or inference exception
app/api.py	Startup model load and successful requests	Invalid request, unavailable model, unexpected error

The error-handling design keeps validation close to the module that owns the risk, text validation in preprocessing, file validation in ingestion, schema and class checks in data loading, model readiness in prediction and API startup, and request validation at the API boundary. This prevents invalid inputs from silently reaching the model while keeping logs useful for debugging without exposing raw resume content.

5. Code formatting and linting

The initial review established a summarized baseline before remediation (see `lint_before.txt`). After formatting, the reproducible evidence is the raw after transcript in `lint_after.txt` plus the live `black/isort/flake8` commands.

```
BEFORE (2026-07-26)
Black: 15 files needed formatting.
isort: 6 files had incorrectly sorted imports.
Flake8: 29 violations.

AFTER (2026-08-15)
$ black --check . -> PASS
$ isort --check-only .
PASS - no incorrectly sorted imports.

$ flake8 .
PASS - no violations.
```

Evidence 1. Recorded baseline and reproducible post-remediation results.

This evidence shows that code style, import ordering, and static lint violations were first measured, then remediated, and finally rechecked using reproducible command-line tools rather than manual inspection.

6. REST API design

The REST API is designed as a thin production boundary over the shared application service. FastAPI owns HTTP routing, schema validation, OpenAPI documentation, and status-code translation, while ResumeScreeningApplication owns resume validation, scoring, explanation, and audit writes. This keeps the API contract stable without duplicating business logic from Streamlit or the ML modules.

Endpoint	Contract	Status behavior
GET /health	Typed status, model_loaded, and API version	200 ready after a successful model load; 503 if the artifact is missing or corrupt
POST /predict and /v1/predictions	Typed candidate and resume request; typed ranking response	200 success; 422 invalid; 503 model; 500 unexpected
GET /metrics	Persisted model/data release snapshot	200 available; 404 snapshot missing
GET /docs	Generated OpenAPI/Swagger documentation	Interactive contract documentation

Validation occurs at both schema and domain layers. A punctuation-only resume is rejected after normalization, blank candidate names are rejected, model loading occurs at startup, and readiness fails when the artifact is missing.

6.1 Runtime evidence for REST API

The screenshots below provide runtime evidence for the REST API contract described in this section. They show the service startup and model load, the generated OpenAPI documentation, health readiness response, interactive prediction request form, successful prediction response, and metrics snapshot.

```
PS C:\Users\prath\Documents\GitHub\Mtech\SEML_Assignment\SEML_Group179_Assignment_II (1)\Group179_Assignment_II_SHARE\Files\179> python run_api.py
INFO: Started server process [35380]
INFO: Waiting for application startup.
2026-08-15 16:37:31.054 | INFO | ml.predictor | Loaded model artifact from C:\Users\prath\Documents\GitHub\Mtech\SEML_Assignment\SEML_Group179_Assignment_II (1)\Group179_Assignment_II_SHARE\Files\179\models\resume_classifier.joblib
2026-08-15 16:37:31.055 | INFO | app.api | API startup loaded model version 2.1.0
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Figure 4. FastAPI runtime startup from `run_api.py`. The console confirms that the application started successfully, loaded the `resume_classifier.joblib` model artifact, registered API version 2.1.0, and began serving through Uvicorn at `http://127.0.0.1:8000`.

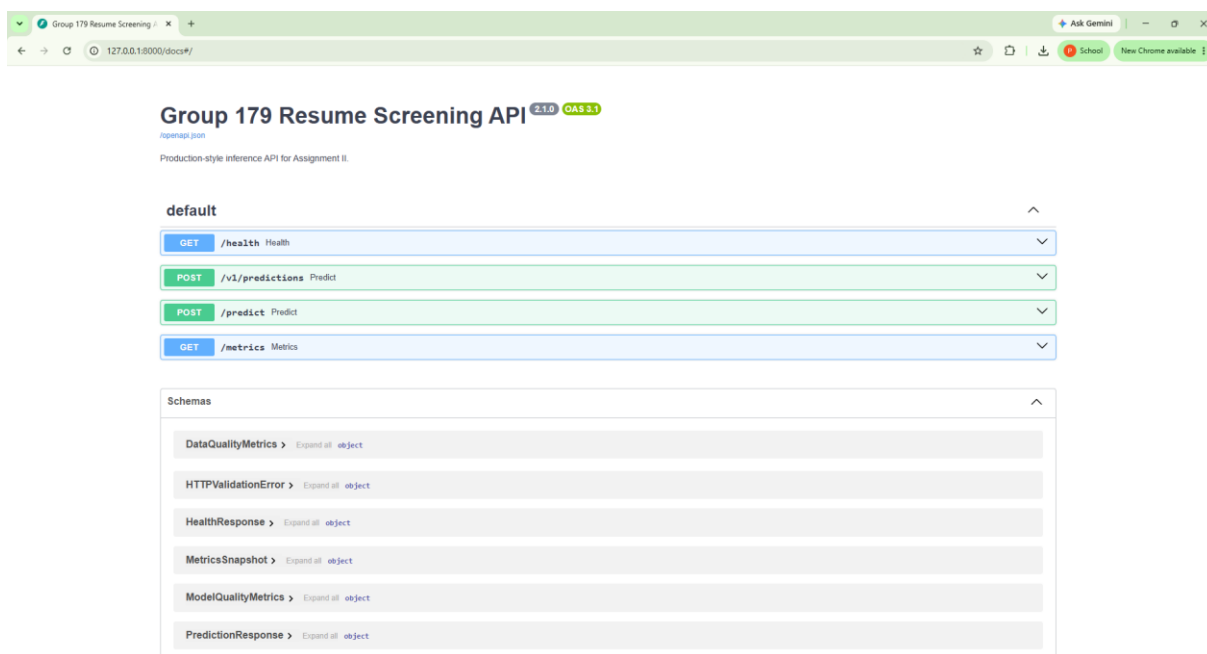


Figure 5. OpenAPI specification - `GET /health`, `POST /v1/predictions`, `POST /predict`, `GET /metrics`, and generated schema models.

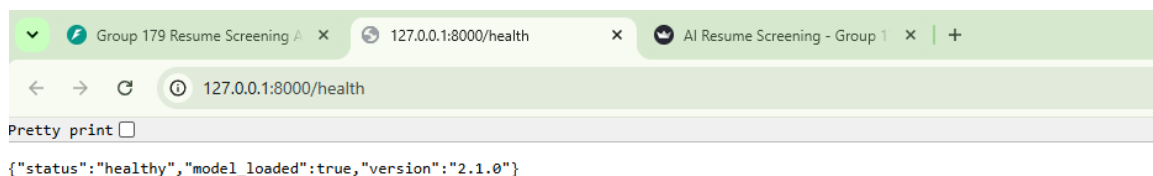


Figure 6. `GET /health` - service ready with `model_loaded=true` and API version 2.1.0.

Parameters

No parameters

Request body required

application/json

Responses

Curl

```
curl -X POST \
  http://127.0.0.1:8000/v1/predictions \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "candidate_name": "Alex Morgan",
    "resume_text": "ALEX MORGAN is a Data Analyst | SQL • Python • Power BI • Excel | Bangalore, India | +91 98765 43210 | alex.morgan@gmail.com | linkedin.com/in/alexmorgan | PROFESSIONAL SUMMARY: Results-driven Data Analyst with 3+ years of experience transforming business data into actionable insights. Skilled in SQL, Python, Power BI, Excel, data cleaning, dashboard development, and statistical analysis. Experienced in partnering with business and operations teams to identify trends, improve reporting, and support data-driven decisions. TECHNICAL SKILLS: Advanced SQL, Python, Pandas, NumPy, Statistics, A/B Testing, UML & Visualization Power BI, Tableau, Excel, DAX, Power Query, Data Cleaning, ETL, Data Validation, API Design, Reporting Tools Jupyter, Git, Postgres, MySQL, Google Sheets, PROFESSIONAL EXPERIENCE: Data Analyst | FinTech Solutions Pvt. Ltd. - Bangalore, India | Jun 2023 - Present • Built and maintained 15+ Power BI dashboards covering revenue, customer retention, operational KPIs, and product performance. • Wrote complex SQL queries and automated recurring reporting workflows, reducing monthly manual reporting effort by 40%. • Analyzed customer and transaction data to identify churn drivers and opportunities, contributing to a 15% improvement in retention within a targeted segment. • Partnered with product and operations teams to define KPIs, validate data quality, and communicate findings to senior stakeholders. • Senior Data Analyst | Retail Analytics Co. - Bangalore, India | Jul 2021 - May 2023 • Cleaned and transformed 500K+ customer and sales records using SQL, Python, and Excel. • Developed automated Excel and Power BI reports that improved weekly performance visibility for regional managers. • Performed sales, cohort, and inventory analysis to uncover underperforming products and regional trends. • Created reusable SQL templates and documentation that improved consistency across recurring analyses. UNSELECTED PROJECTS: Customer Churn Prediction & Analysis: Used Python and SQL to explore customer behavior, engineer features, and build a baseline classification model. Presented churn segments and key drivers through an interactive Power BI dashboard. Website Performance Dashboard: Designed a Power BI dashboard with revenue, margin, YoY growth, regional performance, and product KPIs. Added drill-through views and DAX measures for management reporting. EDUCATION: Bachelor of Technology in Computer Science - ABC Institute of Technology, Bangalore | 2021 | CERTIFICATIONS: Microsoft Certified: Power BI Data Analyst Associate (PL-300) • Google Data Analytics Professional Certificate (2020) | LANGUAGES: English, Hindi, Kannada | Availability: Immediate | Portfolio: github.com/alexmorg
```

Request URL

http://127.0.0.1:8000/v1/predictions

Server response

Figure 7. POST /v1/predictions - interactive request form (candidate_name, resume_text).

Response body

```
{
  "candidate_name": "Alex Morgan",
  "predicted_role": "Data Engineer",
  "confidence": 0.2211897615050818,
  "role_ranking": [
    {
      "role": "Data Engineer",
      "probability": 0.2211897615050818
    },
    {
      "role": "Data Scientist",
      "probability": 0.0816996197530862
    },
    {
      "role": "Frontend Developer",
      "probability": 0.1513709662289577
    },
    {
      "role": "QA Engineer",
      "probability": 0.14015667966187922
    },
    {
      "role": "Backend Developer",
      "probability": 0.1386266367975864
    },
    {
      "role": "DevOps Engineer",
      "probability": 0.1307366248446804
    }
  ]
}
```

Response headers

```
Content-Length: 504
Content-Type: application/json
Date: Sun, 09 Aug 2025 06:28:24 GMT
Server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Example Value | Schema

```
{
  "candidate_name": "string",
  "predicted_role": "string",
  "confidence": 0,
  "role_ranking": [
    {
      "role": "string",
      "probability": 0
    }
  ]
}
```

Figure 8. POST /v1/predictions - successful 200 response with predicted_role, confidence, role_ranking, explanation, and decision_note.

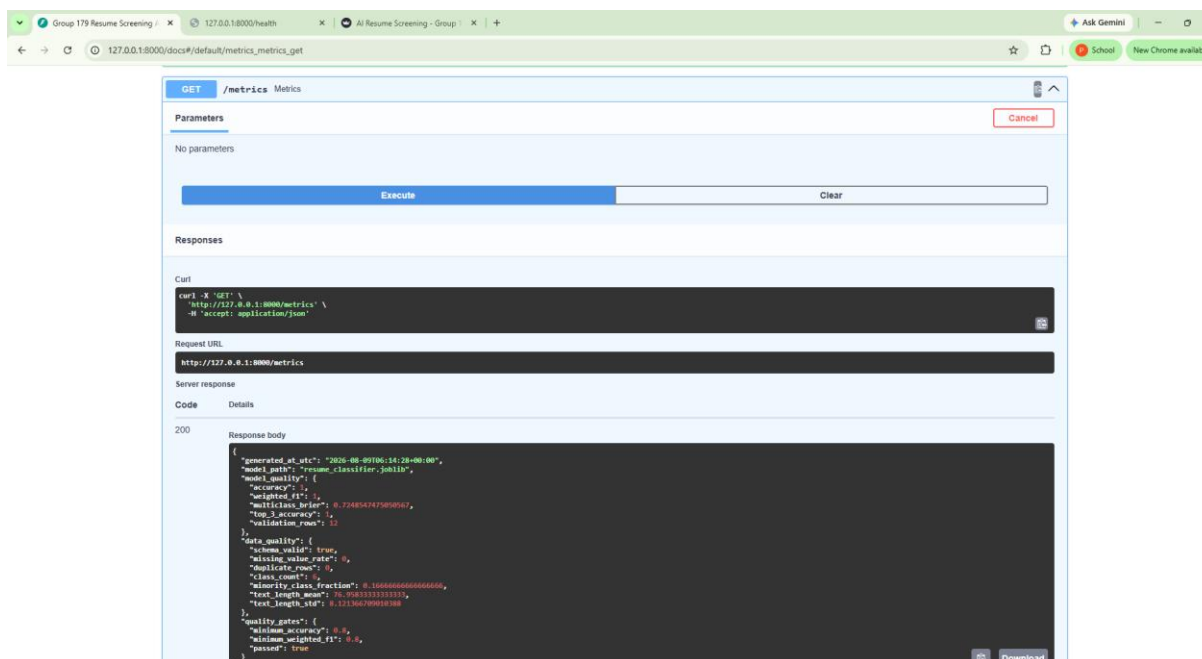


Figure 9. `GET /metrics` - persisted model and data quality snapshot used for release evidence and UI sidebar metrics.

7. Testing strategy and implementation

The test suite is organized around production risk areas rather than only source files. It covers input validation, ingestion continuity, data quality, feature engineering, model training, persisted artifacts, inference contract, API behavior, audit privacy, and monitoring helpers. This ensures that the refactored modules are not only importable, but also preserve Assignment I behavior and enforce Assignment II production-quality expectations.

Type	Representative checks	Evidence
Unit	Normalization, invalid content, TXT ingestion, audit privacy	<code>test_preprocessing.py</code> , <code>test_continuity.py</code>
Data validation	Schema, missing values, duplicates, class coverage	<code>test_data_quality.py</code>
Model training	Loss decreases, gates, determinism, persistence round-trip	<code>test_training.py</code>
Model inference	Ranking/probability contract, direction, invariance, empty input	<code>test_inference.py</code>
API integration	Lifecycle, aliases, metrics, audit privacy, validation/errors	<code>test_api.py</code>
Quality monitoring	Data summary and text-length drift detection	<code>test_quality_metrics.py</code> , <code>test_features.py</code>

```
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\prath\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.13.qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\prath\Documents\GitHub\Tech\SEM4ML_Assignment1\SEM4ML_Group179_Assignment_II (1)\Group179_Assignment_II_SHARE\Files\179
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.0.0
collected 44 items

tests/test_api.py::test_health_endpoint PASSED [ 23%]
tests/test_api.py::test_health_endpoint_reports_corrupt_model PASSED [ 45%]
tests/test_api.py::test_health_endpoint_reports_missing_model PASSED [ 68%]
tests/test_api.py::test_prediction_endpoint_contract PASSED [ 91%]
tests/test_api.py::test_predict_alias_uses_shared_application_layer PASSED [ 114%]
tests/test_api.py::test_prediction_endpoint_persists_privacy_aware_audit PASSED [ 137%]
tests/test_api.py::test_prediction_endpoint_rejects_short_input PASSED [ 160%]
tests/test_api.py::test_prediction_endpoint_rejects_semantically_empty_input PASSED [ 183%]
tests/test_api.py::test_prediction_endpoint_rejects_blank_candidate_name PASSED [ 206%]
tests/test_api.py::test_metrics_endpoint_returns_training_snapshot PASSED [ 229%]
tests/test_api.py::test_metrics_endpoint_returns_404_when_snapshot_missing PASSED [ 252%]
tests/test_api.py::test_api_lifespan_loads_model_once PASSED [ 275%]
tests/test_continuity.py::test_txt_resume_ingestion_preserves_assignment_one_capability PASSED [ 298%]
tests/test_continuity.py::test_pdf_resume_ingestion_extracts_text PASSED [ 321%]
tests/test_continuity.py::test_docx_resume_ingestion_extracts_text PASSED [ 344%]
tests/test_continuity.py::test_unsupported_resume_file_is_rejected PASSED [ 367%]
tests/test_continuity.py::test_audit_repository_stores_metadata_without_raw_resume PASSED [ 390%]
tests/test_data_quality.py::test_data_schema_and_missing_values PASSED [ 413%]
tests/test_data_quality.py::test_invalid_schema_is_rejected PASSED [ 436%]
tests/test_data_quality.py::test_missing_required_values_are_rejected PASSED [ 459%]
tests/test_features.py::test_feature_engineer_fit_transform_shape PASSED [ 482%]
tests/test_features.py::test_feature_engineer_is_deterministic PASSED [ 505%]
tests/test_features.py::test_feature_engineer_handles_unseen_tokens PASSED [ 528%]
tests/test_features.py::test_training_pipeline_uses_feature_engineer_step PASSED [ 551%]
tests/test_inference.py::test_inference_output_shape_and_range PASSED [ 574%]
tests/test_inference.py::test_directional_behavior_for_qa_keywords PASSED [ 597%]
tests/test_inference.py::test_directional_behavior_for_multiple_roles [docker: kubernetes terraform aws cloud automation-DevOps Engineer] PASSED [ 620%]
tests/test_inference.py::test_directional_behavior_for_multiple_roles [react angular frontend typescript css accessibility-Frontend Developer] PASSED [ 643%]
tests/test_inference.py::test_inference_is_invariant_to_case_and_spacing PASSED [ 666%]
tests/test_inference.py::test_inference_rejects_empty_input PASSED [ 689%]
tests/test_preprocessing.py::test_clean_text_normalizes_case_and_spacing PASSED [ 712%]
tests/test_preprocessing.py::test_clean_text_rejects_empty_input PASSED [ 735%]
tests/test_preprocessing.py::test_clean_text_rejects_non_string_input PASSED [ 758%]
tests/test_preprocessing.py::test_clean_text_rejects_input_without_usable_characters PASSED [ 781%]
tests/test_quality_metrics.py::test_data_metrics_measure_completeness_balance_and_length PASSED [ 804%]
tests/test_quality_metrics.py::test_data_metrics_report_invalid_schema_without_raising PASSED [ 827%]
tests/test_quality_metrics.py::test_text_length_drift_rejects_empty_batch PASSED [ 850%]
tests/test_training.py::test_training_creates_model_and_quality_metrics PASSED [ 873%]
tests/test_training.py::test_training_model_can_overfit_small_batch PASSED [ 896%]
tests/test_training.py::test_training_log_loss_decreases PASSED [ 919%]
tests/test_training.py::test_quality_gate_rejects_unusable_model PASSED [ 942%]
tests/test_training.py::test_training_is_deterministic PASSED [ 965%]
tests/test_training.py::test_model_persistence_roundtrip PASSED [ 988%]
```

Figure 10. The screenshot above shows the live pytest execution, while the inventory table below breaks down the same result by test module, function count, collected count, and cases covered. Full verbose transcript: report/pytest_verbose.txt.

Test inventory by module

Module	Fn	Coll	Test cases
tests/test_api.py	12	12	health ready/missing/corrupt; predict contract and alias; audit privacy; short/empty/blank rejection; metrics 200/404; lifespan load
tests/test_continuity.py	5	5	TXT/PDF/DOCX ingestion; unsupported type rejected; audit stores metadata without raw resume
tests/test_data_quality.py	3	3	schema and missing-value report; invalid schema rejected; missing required values rejected
tests/test_features.py	4	4	TF-IDF shape; deterministic names; unseen tokens; pipeline step is FeatureEngineer
tests/test_inference.py	5	6	shape/range; QA direction; two parametrized role cases; case/spacing invariance; empty input rejected
tests/test_preprocessing.py	4	4	normalize case/spacing; empty rejected; non-string rejected; punctuation-only rejected
tests/test_quality_metrics.py	4	4	completeness/balance/length; invalid schema metric; length-drift detect; empty-batch rejected
tests/test_training.py	6	6	gates and snapshot; small-batch overfit; LogisticRegression loss decreases; quality-gate fail; determinism; persistence
Total	43	44	One extra collected item: test_directional_behavior_for_multiple_roles (DevOps, Frontend)

Table 1. 43 test functions in 8 modules; pytest collects 44 items because one inference test is parametrized twice. Full verbose transcript: report/pytest_verbose.txt.

Together, the tests verify both ML behavior and software-engineering behavior: deterministic training, persisted artifacts, robust input rejection, API status contracts, audit privacy, and monitoring helpers. The passing pytest run confirms that the modular refactor did not break the preserved Assignment I workflow or the new Assignment II API and quality requirements.

Evidence location (source package). Automated test output, lint before/after results, and the model/data metrics snapshot are included in the attached source zip (or Git repository) under the report/ folder, alongside this document. Key files: report/pytest_verbose.txt, report/lint_before.txt, report/lint_after.txt, and report/metrics_snapshot.json. Application screenshots are embedded in this report and also stored under Screenshots/ in the source package.

8. Model and data quality metrics

The metrics below are generated by the production training path and persisted in report/metrics_snapshot.json, so they serve as release evidence rather than manually computed report values.

Category	Metric	Measured	Gate / interpretation
Model	Accuracy	1.000	Required ≥ 0.800
Model	Weighted F1	1.000	Required ≥ 0.800
Model	Top-3 accuracy	1.000	Monitoring metric
Model	Multiclass Brier	0.725	Lower is better; monitored
Data	Schema validity	True	Required columns present
Data	Missing-value rate	0.000	Required = 0
Data	Missing required values	0	Required = 0
Data	Duplicate rows	0	Monitored for evaluation bias
Data	Minority-class fraction	0.167	Balanced 6-class set

The evaluation uses 12 stratified validation rows from a deliberately small 24-row synthetic dataset. Accuracy, weighted F1, and top-3 of 1.000 prove that release gates execute; they are not generalization claims. Multiclass Brier of 0.725 with accuracy 1.0 means ranking is correct while probabilities are poorly calibrated (typical top-class confidence is about 0.22-0.32). Brier is therefore a monitoring metric, not a release gate. train_model now calls the text-length drift helper on the train/holdout split; on this demo set drift is not detected.

9. Testing in production and security

This section describes how the system would be tested and protected after deployment. It separates what the submitted implementation already demonstrates from the additional controls that would be required before handling real candidate resumes in production.

9.1 Shadow and canary strategy

A candidate model would receive copied production requests in shadow mode while the current approved model remains the only recruiter-visible output. Shadow results would be used only for comparison, not for decisions. The team would compare latency, error rate, class distribution,

confidence, disagreement, and manually reviewed correctness. Promotion requires disagreement below 10 percent, p95 latency delta below 20 percent, and 5xx rate below 0.5 percent, followed by a 10 percent canary. Automatic rollback fires if any gate fails for 10 consecutive minutes.

9.2 Implemented security and privacy controls

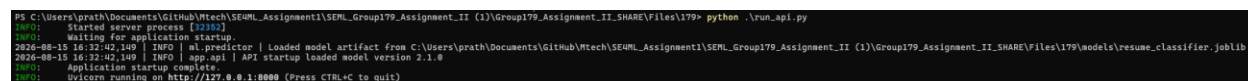
Resume content is sensitive personal data. The submitted implementation reduces exposure by enforcing type, length, semantic-content, file-type, and upload-size validation, avoiding raw resume text in logs; and storing audit metadata without persisting the resume body. Operational monitoring also includes checks for keyword stuffing, input-length drift, confidence shifts, and unexpected role distributions.

9.3 Production security requirements

Authentication and authorization are not implemented in this assignment API and would be mandatory before production use. A real deployment would also require TLS, rate limiting, malware scanning, isolated document parsing, encryption for data and artifacts, retention limits, access auditing, and least-privilege permissions for resume data, audit records, metrics snapshots, and model artifacts.

10. Reproducible execution

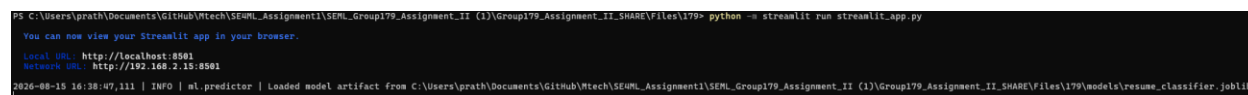
The screenshots below capture the end-to-end local execution of the submitted package, wherein the setup is done and the python code is executing successfully, the subsequent screenshots show the application being executed in local environment.



```

PS C:\Users\prath\Documents\Github\Mtech\SEM_Group179_Assignment_II (1)\Group179_Assignment_II_SHARE\Files\179> python -m run_api.py
[INFO] Started server process [32352]
[INFO] Waiting for application startup.
2026-08-15 16:32:40,109 | INFO | ml.predictor | Loaded model artifact from C:\Users\prath\Documents\Github\Mtech\SEM_Group179_Assignment_II (1)\Group179_Assignment_II_SHARE\Files\179\models\resume_classifier.joblib
2026-08-15 16:32:42,149 | INFO | app.api | API startup loaded model version 2.1.0
[INFO] Application startup complete.
[INFO] Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
  
```

Figure 11. Reproducible command-line setup and execution.



```

PS C:\Users\prath\Documents\Github\Mtech\SEM_Group179_Assignment_II (1)\Group179_Assignment_II_SHARE\Files\179> python -m streamlit run streamlit_app.py
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://192.168.2.15:8501
2026-08-15 16:38:47,111 | INFO | ml.predictor | Loaded model artifact from C:\Users\prath\Documents\Github\Mtech\SEM_Group179_Assignment_II (1)\Group179_Assignment_II_SHARE\Files\179\models\resume_classifier.joblib
  
```

Figure 12. The screenshot demonstrates that the runtime entrypoint starts successfully and can be used to verify the application locally, complementing the API, Streamlit, pytest, lint, and metrics evidence shown in earlier sections.

Application screenshots

The following screenshots demonstrate the running Assignment II system: the preserved recruiter UI (Streamlit) and the production-style FastAPI inference interface required by the assignment submission guidelines.

A. Recruiter application (Streamlit)

The Streamlit app preserves the Assignment I workflow (candidate name, TXT/PDF/DOCX upload or paste, analysis, explanation, and audit history) while routing scoring through the shared modular ResumeScreeningApplication service.

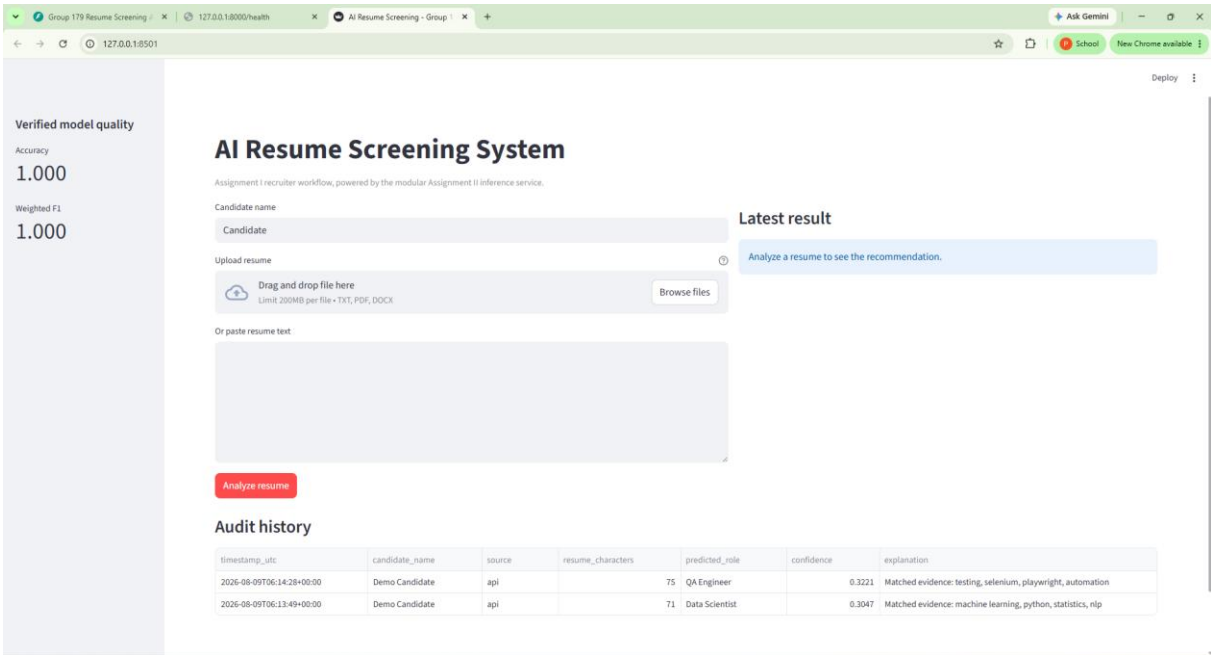


Figure 13. Main recruiter UI - upload/paste controls, verified model quality sidebar, and privacy-aware audit history with no raw resume text stored.

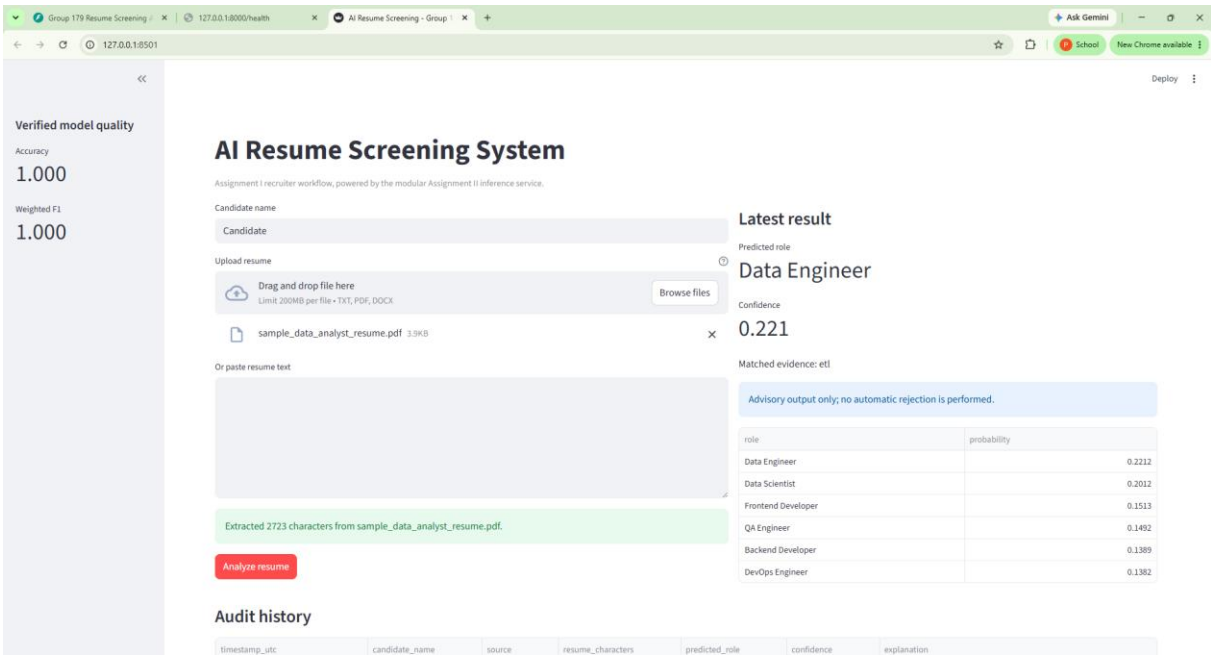


Figure 14. Sample resume analysis - predicted role, confidence, keyword explanation, advisory decision note, and role ranking table.

11. Conclusion

The Assignment I prototype has been evolved into a feature-preserving, production-style ML system. The final package demonstrates explicit research-to-production separation, modular architecture, persistent artifacts, structured logging, robust validation, a typed REST API, multi-level automated testing, quality gates, privacy-aware audit history, and production experimentation planning.

Appendix A - Canonical submission artifacts

The following files form the canonical submission. Earlier draft artifacts remain only as working history and are not required for grading.

Evidence location. Full source code, automated tests (tests/), and quality evidence live in the attached source zip / Git package. The report/ folder holds this Word/PDF narrative together with pytest, lint, and metrics artifacts (pytest_report.txt, lint_before.txt, lint_after.txt, metrics_snapshot.json). UI and API screenshots used above are under Screenshots/ in the same package.

Artifact	Purpose
179.ipynb	Required assignment notebook; research/demo path (production code in ml/, services/, app/)
179.pdf	Submission report (this document); evidence paths described in Appendix
app/, ml/, quality/, services/	Production source package
streamlit_app.py	Preserved Assignment I recruiter workflow
tests/	44 collected items (43 functions)
report/	Taxila report plus pytest, lint, and metrics snapshot in the source zip
models/resume_classifier.joblib	Trained model artifact
README.md and requirements.txt	Setup and dependencies
Screenshots/	UI and API runtime screenshots embedded in Application screenshots section